TWEAG by Modus Create

TWEAG's Proposals for multiple core budget projects for Cardano 2026-2028

This document presents a collective proposal for projects led by Tweag by Modus Create, aimed at advancing the state of the art within the Cardano ecosystem.

It is meant as a starting point for discussing Cardano treasury withdrawals .

As such, we invite anyone interested to comment on the following document, and share feedback with us.

Executive Summary

Vision

Tweag works on the L1 infrastructure for the Cardano ecosystem and prioritizes improving safety and stability of the ecosystem. We embrace full ownership and complete feature delivery. In 2025, most of the work was related to projects that intended to improve the ecosystem; the 2026-2027 proposals focus primarily on ensuring that all the efforts reach mainnet, are used by the community, and are in synergy with other strategic initiatives across the Cardano Ecosystem. The primary growth drivers for Cardano's next phase are the successful deployment and adoption of Peras (finality) and Leios (throughput).

Together, these upgrades enable a fundamental shift in network capacity and usability:

- Higher throughput (Leios) network gets more transactions per second.
- Faster finality (Peras) improves user experience and application design.

With their combined effect we expect increased transaction volume. This leads directly to:

- Increased revenue coming from transaction fees.
- Higher staking rewards.
- Improved network utility and competitiveness.
- Growth in TVL and ecosystem activity.

This proposal focuses on ensuring that this outcome is actually realized in production.
It does so by:

- Delivering Peras to mainnet and advancing its capabilities.
- De-risking protocol behavior under real-world and adversarial conditions.
- Providing tooling and infrastructure required for production adoption.
- Ensuring operational reliability and scalability of the network.

While much of our 2025 effort focused on projects aimed at future improvements, our main

strategic direction for the 2026-2027 proposals is threefold:

1. **Key 2026 Deliverables:** The primary focus for our 2026 proposals is the successful launch of **Peras** to the mainnet, alongside preparing other ongoing projects for the integration of both **Peras** and **Leios**.
2. **Mainnet Delivery and Community Adoption:** Ensuring all our development work reaches the mainnet, is adopted by the community, and is in synergy with other strategic initiatives across the Cardano Ecosystem.
3. **Synergy and Ecosystem Alignment:** We are committed to an approach that avoids silos. All our development efforts are continuously vetted against the ongoing work of Input Output Global (IOG) and other ecosystem contributors to ensure maximum synergy. This includes proactive participation in community governance, technical workshops, and standards discussions to guarantee that Tweag's deliverables are not just technically sound but also strategically aligned with the overall roadmap for Cardano.

For the “Tweag’s core projects 2025-2026” proposal projects that require effort and maintenance we try to provide it during new feature development.

This proposal bundle is intentionally non-modular: 9 packages (split into the 17 work packages) form a single delivery pipeline. Hoarding-node deployments and conformance/mutation/audit testing provide the instrumentation and correctness scaffolding required to validate Peras (v1/v2), history expiry, and block-cost safety changes. Several work packages also require explicit governance and stakeholder actions (Hard Fork/CIP progression), so our success criteria separate (a) vendor-controlled completion from (b) ecosystem adoption and activation.

Where success depends on governance (Hard Fork activation or CIP progression), we commit to “ready-for-activation” deliverables (merged code, reproducible releases, runbooks, benchmarks, and governance-action packages) and separately report on “ecosystem activation/adoption” as a dependent milestone with clear owners and risks.

Team & Budget Administration

About Tweag and Modus Create

At Tweag by Modus Create, we offer a unique combination of deep technical expertise and strategic consulting capabilities, with a long-standing commitment to open-source and decentralized systems. EURL Tweag has been in business for over a decade, having started in 2013, and has built a reputation for engineering excellence across critical infrastructure projects.

Since January 2018, we have been continuously engaged with IOG (Input Output Global) on a

variety of initiatives within the Cardano ecosystem, including core protocol development. Since May 2021, we have extended this engagement to provide formal audits, helping to ensure the reliability and security of mission-critical code.

Our team has played a leading role in the development of Cardano's core infrastructure, including leading the consensus and ledger teams, implementing the Ouroboros Genesis protocol, and contributing to the design of Ouroboros Peras. We have been involved in nearly all aspects of the core Cardano node, giving us a comprehensive understanding of the system's architecture, roadmap, and strategic direction.

Beyond Cardano, we bring deep, practical experience with Haskell, Rust, and a wide range of technologies used in polyglot projects, enabling us to engineer scalable, secure, and high-performance systems across domains.

In 2022 after becoming a part of the Modus Create, a global digital transformation consulting firm, we are backed by a diverse team of strategists, designers, and technologists who help the world's leading brands build digital advantage. Modus Create specializes in strategic consulting, full lifecycle product development, platform modernization, and digital operations and is an official partner of top-tier technology providers such as Atlassian, AWS, and GitHub. This global reach and cross-disciplinary strength provide our clients with unmatched capabilities throughout the full product development lifecycle. In recent years, Modus Create has been heavily involved in AI-transformation, while still preserving the best practices and quality that Tweag is known for.

This proposal reflects our ongoing commitment to advancing Cardano's mission through rigorous engineering, strategic alignment, and high-impact delivery.

Costing and Payments

Administration and Contracting

Tweag by Modus Create requests that Intersect serve as the designated proposal and contract administrator following the roles and responsibilities outlined in the Cardano Constitution. This includes acting as the auditor of record, responsible for assessing progress and verifying that deliverables align with the commitments defined in this proposal.

Tweag anticipates entering into direct contractual agreements with Intersect, with a preference for Milestone Based Fixed Price contract structures where deliverables are clearly defined and scope is limited. Contract types, payment terms, and rate structures with subcontractors will be defined on a case-by-case basis.

Tweag retains full discretion over internal staffing and subcontractor selection, and reserves the right to onboard new or alternative suppliers as needed to ensure successful delivery. While specific security auditors are not identified in this proposal, subcontractors will be responsible for selecting, engaging, and funding security audits required to meet delivery standards.

Multi-Supplier Collaboration

Initiatives involving multiple suppliers will be delivered through close coordination between participating teams, with active engagement and guidance from the Technical Steering Committee (TSC). The TSC will support solution scoping, provide strategic oversight, and help ensure alignment with broader ecosystem priorities throughout the lifecycle of each workstream.

The inclusion of suppliers in this proposal reflects potential collaboration opportunities; however, it does not guarantee that any specific supplier will be engaged for delivery. Contracted suppliers will retain full autonomy over their internal resource allocation, work prioritization, and day-to-day project management responsibilities.

The Software Readiness Levels (SRLs) and release cadence for these initiatives will depend on the outcomes of funding decisions. Tweag anticipates working closely with Cardano development teams to align on release planning, technical milestones, and delivery expectations as proposals are executed.

Project management and contributions

We are dedicated to maintaining transparency and encouraging community contributions as an open-source project. Our commitment includes:

- **Regular Demos :** We aim for a regular demonstration cadence, at least once every two months. All recordings will be stored on the website related to the project https://tweag.github.io/cardano-website/.
- **Status Updates:** Individual project status updates are consistently posted on our website: https://tweag.github.io/cardano-website/.
- **Communication Channel :** A dedicated communication channel for all projects is maintained on the Tweag Discord server. With updates published there on a regular basis.
- **Core Teams Communication :** For projects requiring interaction with core teams, we ensure prompt communication, including advance design discussions, alignment on implementation, detailed explanations, demos, and defining the concrete form of integration on a case-by-case basis. We use a dedicated internal communication channel for projects to support frequent collaboration—both synchronous and asynchronous—enabling quick feedback loops, coordinated decision-making, and smooth teamwork. We also maintain a program-level Slack channel for knowledge sharing and cross-team coordination, ensuring that questions, updates, and expertise flow efficiently across the projects.

Duration & Milestones

The expected duration of all work packages is set for **2 years** to strategically align the total project timeline with the realities of Cardano's mainnet deployment and complex technical scope, specifically to *mitigate* the risk of a full-year delay due to release procedures.

The 2-year duration is necessary for the following reasons:

1. **Strategic Alignment with Hard Fork Cycles (Mitigating Release Delays):** Major protocol upgrades like Peras require a Hard Fork Combinator (HFC) event for deployment. HFC events occur on a fixed cadence. If development is completed too early (e.g., in 9-12 months), the finished product might sit ready for many months, waiting for the next HFC window. By setting the total duration at 2 years, the work can be strategically phased to align the completion of the most advanced features (like Peras v2) with an optimal HFC target, thereby minimizing the time the final deliverable spends waiting for network activation and **preventing a potential long delay.**
2. **Staggered Delivery and Dependency Management:** The document highlights that the more advanced **Peras v2** work is dependent on the successful launch and initial operation of **Peras v1 ready for the mainnet**. A 2-year plan provides the necessary time for this crucial *staggered delivery* sequence, where the foundational v1 work stabilizes before the v2 extensions are finalized and released.
3. **Comprehensive Scope and Non-Development Requirements:** The proposal includes 17 distinct work packages spanning protocol changes (Peras, Leios), network reliability (Hoarding Node, History Expiry), and critical developer tooling. The 2-year timeline accounts for the cumulative effort, mandatory security/protocol audits, and extensive integration required for such a broad suite of mutually reinforcing projects to be delivered to a mainnet-ready standard.
4. **Immediate Start for Peras Extensions:** Crucially, the project structure ensures that planning and **implementation for Peras extensions (Peras v2) start immediately and run in parallel** with the finalization of Peras v1. The longer total duration only applies to the final deployment and comprehensive integration, not the initial development phases.

Total Budget Ask

The total ask is **₳39,787,316.00** (covering a USD budget of **$9,946,829.00**) for 2026-2028 years. The full budget breakdown is given below.

All scopes are estimated in *FTE* (Full-Time Equivalent) based on an average hourly market rate. Having now spent a year working with contractors and service providers in our ecosystem, we have arrived at an average hourly rate of around $176, which gives us a figure of **$10M** spread over the two year period.

We (still) use a conservative conversion rate of **0.25 ADA** [₳] per **USD** [$]. This rate is based on an average price calculation over the past 5 years, resulting in a price range of $0.19 to $0.25.

We provide a single portfolio budget table listing each work package total and a reconciliation checksum confirming that the sum equals **₳39,787,316.00**. Any “fixed price /prior-cycle” line item is labelled with what it buys in 2026–2027 and why it is included in this ask.

Project	Budget ask (₳)
Peras v1 ready to the mainnet	₳ 4,954,688.00
Peras v2 improvements and resilience	₳ 8,240,040.00
History Expiry	₳ 2,816,016.00
Hard Fork Incident Mempool Bridger	₳ 1,478,408.00
Conformance Testing of Consensus Peras and Leios	₳ 4,860,760.00
Conformance Testing of Consensus Generate forks from adversarial branches	₳ 1,408,008.00
Canonical ledger state maintenance and Mithril integration	₳ 2,464,014.00
Plutus Script Re-Executor ecosystem integration	₳ 6,724,024.00
Mutation Testing for Cardano	₳ 1,408,008.00
Hoarding Node Extensions: Live deployment	₳ 2,969,336.00
Hoarding Node Extensions: Distributed mode	₳ 469,336.00
Hoarding Node Extensions: Embedded Consensus Validation of Blocks and Headers	₳ 469,336.00
Hoarding Node Extensions: Transaction Collection	₳ 352,002.00
Block Cost Investigation Extensions	₳ 469,336.00
Cardano-node-emulator maintenance	₳ 469,336.00
Genesis-Sync-Accelerator	₳ 234,668.00
Total	**₳ 39,787,316.00**

Alignment with the Strategy Framework

Strategic pillar

Pillar 1: Infrastructure and Research Excellence
Pillar 5: Ecosystem Sustainability and Resilience

Our main goal with this proposal is to supercharge the fundamental **L1 protocol infrastructure** and make it more reliable for everyone. We've chosen to focus heavily on **Infrastructure and Research Excellence** because building a rock-solid foundation is the most critical step for Cardano's future growth and widespread adoption. Our approach delivers a comprehensive set of mutually-reinforcing software tools, centered around improving the **Peras delivery mechanism**. This concerted effort not only strengthens the core but also directly contributes to **Pillar 5: Ecosystem Sustainability and Resilience**, ensuring the network can handle future challenges and maintain its long-term health.

Work packages description

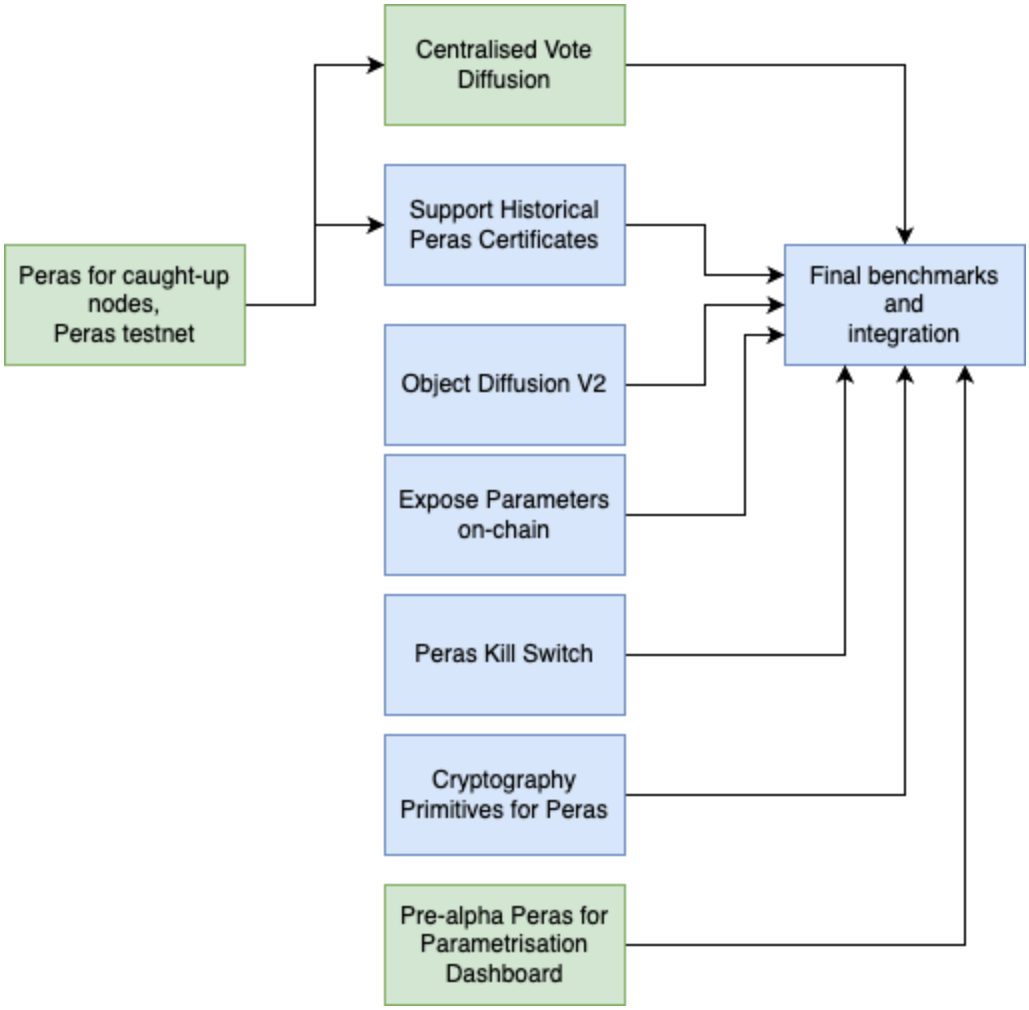
Peras v1 ready for the mainnet

Area: Adoption / L1 protocol improvements
Core KPI: Transaction per day, TVL

High level description

Peras is a core protocol upgrade that significantly improves transaction finality on Cardano, reducing settlement time from approximately 12 minutes to a target of 2–5 minutes under normal conditions. This improvement is essential for enabling responsive applications, efficient L2 solutions, and increased transaction throughput utilization. As part of the broader scaling roadmap alongside Leios, Peras is a primary driver of increased transaction volume and economic activity on the network. This work package focuses on delivering all remaining functionality required to bring Peras v1 to mainnet readiness, ensuring it is secure, operable, and usable by node operators and developers.

In 2025, we proposed to work on the following three tasks (colored in green) for Peras. Their relation and dependencies to each other and the remaining work is depicted below. In this proposal we are going to work on the tasks that are left for preparing Peras v1 for the preprod environment (colored blue).



The purpose of this work package is to provide all missing functionality for pre-alpha-Peras to be deployed on mainnet. This version is called Peras-v1. The required parts of the implementation are

- Support of the historical certificates — this is a part of the protocol that affects the bootstrapping phase and protects from the several kinds of attacks that may lead to a fresh node choosing the wrong chain before it has caught up with the rest of the network.
- Cryptography primitives — pre alpha Peras does not focus on using real cryptographic schemes. As a part of the delivery to the mainnet, we will implement and apply real cryptography schemes as well as run required benchmarks to ensure sustainability of the solution.
- Kill Switch implementation — as a part of disaster-recovery procedures, we plan to introduce a way to coordinate disabling and re-enabling of Peras on demand. It includes changes in configuration,API, and documentation for SPOs.

Core objectives

- Implement production-grade cryptographic primitives and integrate them into the Peras protocol.
- Support historical certificates required for secure node bootstrapping and chain selection.
- Introduce a KillSwitch mechanism enabling coordinated activation and deactivation for operational safety.
- Expose all tunable Peras parameters on-chain for transparency and governance control
- Extend the cardano-cli to support certificate operations, including generation, inspection, and validation.
- Deliver developer and SPO documentation with example workflows for certificate management.
- Enable local simulation of Peras behavior and certificate flows for testing and validation.

Expected Value

Faster and more predictable settlement is important for L2 solutions and partner chains, which rely on timely and reliable L1 confirmation for their operation. While such systems can operate under current conditions, improved finality may significantly enhance their efficiency and usability.

Peras improves settlement under non-degraded conditions, achieving approximately 2 minutes finality with overwhelming probability under standard assumptions, compared to approximately 12 minutes with Praos (in good conditions). This enables more responsive interactions across DApps, L2 solutions, and partner chains (e.g. Midnight), allows applications to increase interaction frequency, directly increasing transaction volume per user (**Monthly transactions**) and improved user engagement (**MAU**), as applications can offer more responsive interactions. Faster and more predictable settlement may also strengthen confidence in DeFi and L2 systems, potentially contributing to **TVL** growth.

Peras does not increase throughput capacity directly, but may improve utilization of existing capacity. Its additional resource overhead is modest, providing a favorable tradeoff between performance and cost.

Metrics for Success

- Peras is deployed and activated on Cardano mainnet via a completed hard fork, success of these metrics depends on the governance action and is outside of the vendor control.
- Median settlement time is ≤ 5 minutes under non-degraded network conditions and ≤25% adversarial stake conditions, compared to baseline 12 minutes.
- Storage usage increases by no more than 30% compared to Praos under non-degraded network conditions and ≤25% adversarial stake.
- No emergency disablement (KillSwitch activation) occurs during the first 6 epochs (~30 days) after mainnet activation.

"**Settlement time**" is time from transaction submission to reaching the proposal's defined settlement criterion (must specify: k blocks / ledger confirmations / finality rule).

"**Non-degraded network conditions**" = a published network profile (e.g., connectivity threshold, latency distribution, packet loss envelope) used consistently across benchmarks.
"**Observation period**" = a specified continuous window with start/end dates (or epoch range) and a published measurement pipeline.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

Project continuation

Peras code will still be maintained, but it should be done as parts of the following projects.

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T1.1 Project Kickoff	Public repository and tracking board (issues, milestones). High-level architecture overview of Peras v1 components. Recorded public kickoff demo.	Repository and tracker are publicly accessible. Project plan overview document is published. A recorded demo is available online on the project website.	1
T1.2 Finalized Requirements and Implementation Plan	Complete requirements specification including but not limited to: - KillSwitch design. - On-chain parameters. - Historical certificates. - Cryptographic requirements. - Performance requirements. Public implementation plan mapped to components.	The requirements document is complete and publicly accessible. Each component is traceable to requirements. The implementation plan is published and reviewed. Cryptography requirements handed off to the research team.	4

T1.3 Peras v1 Functional Implementation (Pre-Mainnet)	Implemented: - KillSwitch mechanism. - On-chain parameter storage. - Certificate handling tools. - Network integration updates. Integrated into the node codebase (with placeholder cryptography). *For the cryptographic scheme BLS scheme is used, but it may be changed based on the result of the research.*	All components are accepted for merge into relevant repositories. End-to-end functionality demonstrated in the test environment.	16
T1.4 Production Cryptography Integration	Integration of finalized cryptographic primitives. Support for historical certificates using production crypto, in case it is required by the research. *This work depends on outcomes of the research team.*	Cryptographic implementation replaces placeholder solutions. All related components are integrated and functional. End-to-end functionality demonstrated in the test environment.	8
T1.5 Mainnet Release Preparation and Hard Fork Readiness	Code merged into release branch. Deployment on pre-production network. SPO documentation and operational guides. Governance actions prepared.	Peras is included in the hard fork candidate codebase. Successfully deployed and tested on preprod. Documentation is publicly available. No critical blocking issues remain.	8

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development (Labour)	FTE	32	₳ 117,334	₳ 3,754,688
Research and development work on security by CBU	Work package and support	1	₳ 1,200,000	₳ 1,200,000

Peras v2 improvements and resilience

Area: Adoption / L1 protocol improvements
Core KPI: Transaction per day, TVL

High-level overview

The Peras-v1 implementation will establish an initial baseline for improved finality on mainnet. However, the following improvements will still be needed to reach a complete implementation as originally envisioned by the Peras researchers:

- **Avoiding cooldown by pre agreement** — one of the drawbacks of Peras-v1, is that it takes quite conservative path in case a voting has failed. In this case Peras algorithm stops for X until it would be safe to restart. There is a theoretical attempt to solve this problem in the bizantinne scenario by introducing a pre agreement algorithm that consists of pre-vote, byzantine agreement and vote phases. And allows control of the probability of the voting to succeed by controlling the length of byzantine agreement. This approach allows to avoid cooldown entry in case an adversary node has less than 25% of stake. For this project we are planning to do experiments for finding the optimal parameters for the preagreement to avoid cooldown and implement a prototype solution to allow testing on a testnet.
- **Faster cooldown recovery** — while pre-agreement would allow us to reduce the probability of entering a cooldown period, doing so is sometimes inevitable in the presence of a sufficiently powerful adversary with about 25% of the total stake. , In this part of the project we aim to explore alternative paths to exit a cooldown period faster without compromising the Praos safety property.
- **Engineering improvements over Peras-v1** — having deployed Peras-v1 to mainnet, we can collect performance metrics and apply data-driven optimizations such as:
    - Low-level Node to Node optimizations — the best approach to exchanging Peras certificates and votes over the wire depends on several dynamic characteristics present on mainnet (stake distribution, network topology, etc.). We aim to introduce optimizations in several layers of the communication stack (network mini-protocols, low-level codecs, etc.) to improve the throughput and bandwidth of w.r.t. Peras-v1. In particular, we intend to revisit the ObjectDiffusion protocol and specialize it to the specific dynamic of certificate diffusion and the one of vote diffusion for minimal footprint in each case.
    - Node memory and CPU footprint — Peras-v1 takes a conservative approach to the introduction of complex data structures and algorithms, valuing correctness over performance. We aim to profile the existing implementation in search for hot spots where a more suitable data structure or algorithm could be beneficial to improve the node's performance. In addition, we will revise the assumptions taken conservatively by the current implementation and evaluate if they could be safely relaxed, allowing for further optimizations to be introduced.

- Committee selection scheme — Peras-v1 will initially use a weighted Fait-Accompli committee selection scheme to compute a semi-fixed size committee of pools that are entitled to vote for blocks to be boosted on every Peras round. This scheme aims to provide a fair split between fairness among pools and resilience against attacks. However, certain low-lever implementation details such as the concrete cryptographic scheme used to validate eligibility of nodes can severely hinder its applicability due to an increased overhead in vote and certificate size and validation speed. We aim to evaluate alternative committee selection schemes and parameterizations to find the one that works best in practice.

Core objectives

- Implementation of the new algorithms related to preagreement if approved by the Research team.
- Engineering improvements of the Peras implementation: such as choosing a more efficient approach for the committee selection if proven to be more simple and efficient.
- Extend the Peras Dashboard to expose pre-agreement parameters and runtime behavior.
- Publish experiment results through static dashboards illustrating cooldown probability and recovery characteristics.
- Enable reproducible experimentation by providing tooling to run Peras parameter simulations locally.

Expected value

After bridging Peras to mainnet, further improvements are required to ensure reliable operation under real-world conditions. Peras v2 enhances the consistency and resilience of finality by introducing pre-agreement mechanisms that reduce the likelihood of failed voting rounds. This minimizes time spent in cooldown periods, where finality is temporarily unavailable, and improves recovery under adverse conditions.

By making finality more reliable and predictable, Peras v2 improves usability for DApps, L2 solutions, and partner chains, which depend on stable and timely settlement. These improvements support sustained transaction activity (**Monthly transactions**) and may contribute to increased user engagement (**MAU**) by reducing disruptions and improving user experience. More reliable settlement may also strengthen confidence in DeFi and L2 systems, potentially contributing to **TVL** growth.

While Peras v2 does not increase throughput capacity directly, it improves utilization of existing capacity by reducing interruptions and inefficiencies in the consensus process. It also strengthens operational reliability by reducing periods where finality is unavailable, supporting high network availability and production-grade behavior, and providing a better tradeoff between performance, resilience, and resource usage.

Metrics of Success

- Peras remains in active (non-cooldown) phase for ≥ 90% of total enabled time over a 3-month observation period on mainnet.
- Storage usage increases by no more than 30% compared to Praos under non-degraded network conditions and ≤25% adversarial stake.
- P95 settlement time is ≤ 5 minutes under non-degraded network conditions and ≤25% adversarial stake, measured over a 3-month observation period on mainnet.
- Recovery time from cooldown to active phase is measured and reported, including distribution (median, p95) over the observation period.

"**Settlement time**"—is time from transaction submission to reaching the proposal's defined settlement criterion (must specify: k blocks / ledger confirmations / finality rule).

"**Non-degraded network conditions**"—a published network profile (e.g., connectivity threshold, latency distribution, packet loss envelope) used consistently across benchmarks.

"**Observation period**"—a specified continuous window with start/end dates (or epoch range) and a published measurement pipeline.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T2.1 Network Model & Experimentation Framework	Network model capable of simulating: - varying stake distributions adversarial behavior (≤25%). - different parameter configurations. Experimentation framework for running controlled scenarios. Public demo of simulation results.	The simulation framework is publicly available and runnable. Results are reproducible from published instructions. Public demo recording is available on the project website.	8

T2.2 Pre-Agreement Algorithm Evaluation	Experimental evaluation of pre-agreement approach: - Impact on cooldown frequency. - Impact on latency and resource usage. Comparison with baseline (Peras v1). Recommendation document (GO / NO-GO). *Deliverables of this milestone may require a subcontract to CBU.*	The report includes quantitative comparison vs baseline. Trade-offs (performance, complexity, risks) are explicitly documented. Recommendation is reviewed with research/consensus teams. The decision is publicly documented.	8
T2.3 Pre-Agreement Prototype (Conditional)	Prototype implementation of pre-agreement mechanism. Deployment on a private testnet or controlled environment. Performance measurements vs Peras v1.	Prototype is integrated into nodes or test framework. Demonstrated reduction in cooldown frequency OR improved recovery. Results are reproducible and publicly documented. Demo of behavior under adversarial scenario with recording available on the project website.	12
T2.4 Cooldown Recovery Improvements	One or more recovery strategies implemented. Benchmarking against baseline recovery behavior.	Demonstrated improvement vs Peras v1 baseline OR justified trade-off. Implementation is accepted for merge into codebase OR documented as rejected approach.	10
T2.5 Performance Optimization of Peras	Optimizations in: - networking layer (miniprotocol design and message encoding) - memory/CPU usage. - committee selection mechanism. Profiling reports identifying bottlenecks.	Benchmarks show measurable improvement in at least one: - bandwidth usage. - CPU usage. - memory footprint. No regression in correctness or consensus behavior. Changes are accepted for merge in the relevant repositories.	10

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost

Development	FTE	60	₳ 117,334	₳ 7,040,040
Research and support by CBU/ARK		1	₳ 1,200,000	₳ 1,200,000

History expiry

Areas: Reliability
Core KPI: Alt. Full Node Clients, Scaling the L1 engine

High-level description

As transaction throughput increases with the introduction of Leios and improved finality via Peras, the storage requirements for Cardano nodes are expected to grow significantly. Without intervention, this growth risks making node operation economically unsustainable, reducing participation and weakening decentralization. *History Expiry* introduces support for partial-history nodes, allowing nodes to operate without storing the full historical blockchain while preserving security guarantees through the full header chain. This work is essential to ensure that increased throughput does not translate into prohibitive infrastructure costs.

The ultimate goal is to enable a flexible, cost-effective spectrum of node types — from lightweight nodes with limited history to full archival nodes — without compromising the network's security or decentralization principles. This evolution is essential for ensuring the long-term economic viability and scalability of the Cardano staking infrastructure.High transaction volume (100–1,000+ TPS) from the Peras and Leios integrations will cause Stake Pool Operator (SPO) disk usage to surge, potentially by ~1 GB/hour. Requiring SPOs to store the **entire** historical blockchain is becoming too expensive and economically unsustainable.

Proposal: Implement an optimized data storage solution, similar to Ethereum's, to allow for "partial history" nodes. This decouples daily node operation from the high cost of full archival storage, a concept already proven on the mainnet (e.g., Amaru). The idea is to keep the full **Header chain**, while the **block chain** will be truncated to a certain depth. The core idea of the project is to ensure that the Cardano network will continue being effective and operational with partial nodes available on the network.

This solution introduces flexible, cost-effective nodes, ensuring the long-term economic viability and scalability of Cardano's staking infrastructure. This guarantees network health without compromising security or decentralization, as all nodes will still securely maintain the full transaction *Header Chain* history.

Core objectives

- Define and submit a Cardano Improvement Proposal (CIP) enabling nodes to operate without storing full historical blockchain data while preserving security guarantees.
- Specify network-level configuration parameters for safe operation of partial-history nodes (e.g., minimum history window, protocol behavior).
- Deliver a production-ready implementation of a Cardano node supporting partial history storage.
- Provide operational documentation and tooling for operators to safely migrate, validate, and monitor partial-history nodes with provided tooling.

Expected Value

The proposed solution reduces storage requirements for node operators by enabling partial-history nodes, lowering the cost of participation for SPOs. As transaction throughput increases, continuous growth of historical data risks becoming a limiting factor; this proposal prevents storage from turning into a bottleneck and improves the economic sustainability of running nodes.

By decoupling node operation from full historical storage, the proposal enables long-term scalability of the L1 engine, allowing higher throughput without imposing unsustainable infrastructure costs (**Throughput capacity per day**). Lower operational requirements may also improve network reliability and decentralization by enabling broader participation from node operators.

These improvements may indirectly support increased transaction activity (**Monthly transactions**), user engagement (**MAU**), and confidence in the ecosystem (**TVL**), by providing a scalable and economically sustainable infrastructure.

Metric of Success

- Percentage of active nodes operating in partial-history mode on mainnet.
- Storage growth per node is reduced compared to full-history nodes under equivalent transaction load.
- Partial-history nodes remain fully compatible with consensus, with no increase in fork rate or validation errors compared to baseline.

Classification

New initiative or continuation of existing: New initiative
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T3.1 Project Kickoff	Public repository and documentation hub. Problem statement and scope definition. Initial design outline for partial-history nodes. Recorded public kickoff/demo.	Repository and documentation are publicly accessible. Demo recording is publicly available on the project website.	2
T3.2 Protocol Specification for Partial-History Nodes (CIP)	Establish a clear consensus on the problem and constraints with the community. Specify what it means for a node not to have the full history: - Mini-protocols: changes to ChainSync to let nodes communicate to partial nodes - Define a minimum history window (e.g. 5 epochs). - Re-affirm the header chain requirement: nodes should have the full header chain to validate blocks.	CIP draft is published and submitted, reaching 'Proposed'. Specification fully and unambiguously defines node behavior. Feedback from consensus/network teams is incorporated. No critical open design questions remain.	4
T3.3 Node Implementation with Bounded History Storage	Implementation of: - sliding window storage model. - protocol changes from CIP. - updated validation and replay logic. Integration with existing node components.	Node runs using bounded history (no full chain required). Functional tests pass under normal operation. Crash/restart recovery strategy is implemented. Implementation reviewed and accepted by relevant teams.	8
T3.4 Ecosystem integration	Align on the integration with Mithril and GSA decide on and implement requirements. Prepare documentation resources for SPOs (configuration, storage requirements, tradeoffs). Deployment guide.	Documentation is complete, and accessible for reference. Documentation is published on Cardano community resources. A third party (external operator) can run a node using only the documentation. No missing critical steps.	8

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Delivery (Labor)	FTE	24	~~₳~~ 117,334	~~₳~~ 2,816,016

Supporting resources

Full proposal: https://drive.google.com/file/d/1nEuvhOsAX7VPgFJajz86AeOE4hP_Bd8j/view

Hard Fork Incident Mempool Bridger

Area: Adoption / L1 protocol improvements
KPI: Monthly uptime

High level description

This proposed tool is intended to make that outcome more thorough and more reliable. Accidental hard forks pose four main risks:

- **Manual Recovery:** Protocol disagreement can lead to nodes being unable to automatically rejoin the network, requiring manual intervention, which delays honest stake from coming back online and temporarily weakens security.
- **Double Spend:** Degraded settlement times allow adaptive attackers a higher chance of successful double-spend attacks before the incident is resolved (rectified).
- **Orphaned Block Rewards:** Block issuers on orphaned chains lose their rewards when all nodes eventually switch to the final, preferred chain.
- **Orphaned Block Transactions:** Transactions in orphaned blocks are not guaranteed to be included in the blocks of the final preferred chain.

The proposed tool directly mitigates the **Orphaned Block Transactions** risk and partially and indirectly mitigates the **Double Spend** risk.

Its core value is ensuring transaction flow even between nodes that would normally disconnect during a hard fork incident (e.g., due to one validating a block the other considers invalid). This is achieved by allowing vigilant community members to deploy the tool upon recognizing an incident.

The tool aims to ensure that, unlike during past incidents, transactions from orphaned chains are more thoroughly and reliably included in the final, resolved chain, protecting against the loss of transactions.

Core Objectives

- Implement a tool that enables transaction propagation across nodes during hard fork incidents, including between nodes on conflicting chains.
- Ensure preservation and re-inclusion of transactions from orphaned chains into the final canonical chain.
- Reduce recovery time and improve network safety during fork incidents by minimizing transaction loss and manual intervention.
- Provide operational tooling and documentation to allow node operators and community members to deploy and use the solution during incidents.
- Provide a docker image with the tooling and runbook.

Expected Value

The proposed solution reduces the impact of hard fork incidents by enabling transaction propagation across conflicting chains, ensuring that transactions included in orphaned blocks are not lost. This improves the integrity and continuity of transaction processing during network disruptions.

By minimizing transaction loss and reducing the need for manual recovery, the tool shortens recovery time and improves operational reliability (**Monthly uptime**). It also helps preserve transaction activity during incidents, supporting consistent transaction throughput (**Monthly transactions**).

By improving reliability and reducing the risk of lost transactions, the proposal may strengthen user and developer confidence in the network, potentially contributing to broader adoption and ecosystem stability.

Metrics of Success

- ≥ 80% of transactions from orphaned forks are successfully re-included in the canonical chain after recovery, as measured in controlled fork scenarios on a private testnet.

Classification

New initiative or continuation of existing: New initialive
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T4.0 Project Kickoff	Public repository with initial project structure. Documented problem statement and solution outline. Public communication describing the tool and its purpose. Recorded demo or walkthrough introducing the project.	The repository is publicly accessible with initial documentation. Problem statements and intended solutions are clearly described. A public demo or walkthrough is available online on the project website.	2
T4.1 Mempool bridger Design Specification	Draft design describing how transaction propagation across conflicting chains will work. Refined and finalized design incorporating feedback from consensus team. Clear description of system behavior during fork scenarios.	The design document is reviewed by the Consensus Team. All major design questions are resolved or explicitly documented. The final design version is published and accessible.	6
T4.2 Minimal Working Implementation for Incident Scenario	Initial working version of the tool capable of: - propagating transactions across divergent peers. - operating during a simulated fork scenario. Demonstration setup including transaction filtering capability.	Tool successfully demonstrates transaction propagation in a controlled fork scenario. Demo includes observable handling of transactions across conflicting chains. Functionality is reproducible based on provided instructions.	10
T4.3 Production-Ready Multi-Peer Implementation	Extended implementation supporting: - Multiple peer connections. - Dynamic (hot-swappable) transaction filtering. Finalized instructions for reproducing full system behavior.	Tool operates with multiple peers simultaneously without failure Filter logic can be modified without restarting the system End-to-end behavior is demonstrated and reproducible. Implementation is reviewed by the Consensus Team.	4
T4.4 CIP Submission	Draft Cardano Improvement Proposal (CIP) for TxExport or equivalent mechanism.	CIP is formally submitted to the CIP repository.	4

	Iterative updates addressing community and reviewer feedback.	Review feedback is addressed through documented revisions. Proposal reaches at least “Proposed” status or equivalent review stage.	

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development	FTE	12	₳ 117,334	₳ 1,408,008
Consensus team review	hrs	100	₳ 704	₳ 70,400

Supporting resources

Full proposal: https://drive.google.com/file/d/1wPp6jfPfwajUIlEGnOSrw24Lj9flaXdy/view

Conformance Testing of Consensus: Peras and Leios

Area: Operational resilience / Reliability
Core KPI: Monthly uptime, Alt. Full Node Clients

High level description

This proposal extends the Conformance Testing of Consensus (CTC) framework to support emerging consensus features while ensuring its reliability and usability. The CTC framework represents an initial investment toward a shared, implementation-agnostic conformance testing infrastructure for the Cardano consensus protocol; while still maturing, it establishes the foundations for validating consensus behavior across independent node implementations. This proposal consolidates that foundation by extending scenario generators and improving the testing infrastructure to account for the features of Ouroboros Peras and Ouroboros Leios.

Core Objectives

- Integrate the Conformance Testing of Consensus (CTC) framework into the Cardano ecosystem and core repositories. It includes providing required executables and developer documentation that can be used to perform all required testing.
- Extend the CTC framework to support Ouroboros Peras consensus features.
- Extend the CTC framework to support Ouroboros Leios consensus features.
- Define and implement executable conformance properties that validate protocol guarantees for both Peras and Leios.

Expected Value

This project enables validation of emerging consensus mechanisms under realistic and adversarial conditions prior to deployment. By extending the CTC framework to support Peras and Leios, it reduces implementation risk and increases confidence in protocol correctness across node implementations.

This supports safer protocol upgrades and more reliable network operation (**monthly uptime**). By enabling independent clients to verify conformance, it also contributes to **operational resilience** by reducing single-client risk.

Indirectly, by enabling safer adoption of Peras and Leios, the project supports scalability and ecosystem growth, **contributing to throughput capacity per day** and **TVL**.

Metrics of Success

- Executable conformance tests for Peras and Leios are implemented and successfully executed against at least one node implementation.
- At least one alternative node implementation integrates CTC tests into its CI pipeline, producing a test results log/report.[1]

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T5.1 CTC Integration into Cardano Ecosystem	Integration of the Conformance Testing of Consensus (CTC) framework into Cardano repositories.	CTC framework is merged into relevant Cardano repositories. Tests can be executed within the standard development or CI workflow. Documentation is available describing usage and integration.	4

[1] As Tweag does not have control over alternative node code, both metrics will be reified as a public demo showing a test run against an alternative implementation.

	Alignment with existing development workflows and tooling. Documentation describing how CTC is used within the ecosystem.		
T5.2 Executable Consensus Tests for Peras	Extended CTC generators capable of modelling Ouroboros Peras features. Executable properties validating Peras consensus behavior, ensuring correct handling of voting, certificates, chain selection, and protocol fallback conditions.	Code is reviewed and accepted for merge into the relevant repositories. Tests execute successfully and validate expected consensus properties. Public demo demonstrates test scenarios and results.	12
T5.3 Executable Consensus Tests for Leios	Extended CTC generators capable of modelling Ourobors Leios features. Executable properties validating Leios consensus under its timing constraints, ensuring correct behavior under stochastic certification outcomes and protocol-specific dynamics.	Code is reviewed and accepted for merge into the relevant repositories. Tests execute successfully under defined scenarios. Public demo demonstrates test scenarios and results.	24

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Implementation of the CTE framework	Fixed price	1	₳ 2,500,000.00	₳ 2,500,000.00
Development (Labour)	FTE	20	₳ 117,334	₳ 2,346,680.00
Reviews of the consensus team	Hours	20	₳ 704	₳ 14,080

Supporting resources

Full proposal: https://drive.google.com/file/d/1gz8XpH1vo1iXXW8rYOXx_MkvX27l4b7H/view

Conformance Testing of Consensus: generate forks from adversarial branches

Areas: Reliability
Core KPI: Alt. Full Node Clients, Scaling the L1 engine

High level description

Property based testing (PBTs) test software by feeding it diverse, random inputs. The current input generator for the Genesis protocol's block tree structure is limited. Specifically, it doesn't model "attack" scenarios where adversarial branches in the block tree fork off *other* adversarial branches, potentially leading to undiscovered vulnerabilities. In addition the tests are non-deterministic and sometimes lead to false negatives, such tests are thought flaky and results are ignored. It increases CI time and may lead to the bugs being missed.

The Solution:
We will enhance the tree generator to produce these more diverse and challenging block tree structures. This involves adding a generation phase to create less-dense (adversarial) branches that fork from existing adversarial branches. Once the more realistic test trees are generated, we will update the necessary components, starting with the point schedule generator, which dictates block transmission timing in the simulation. By running the PBTs with these enriched inputs, we expect to find and fix potential protocol failures, ultimately making the Genesis implementation safer.

Core Objectives

- Extend the Conformance Testing of Consensus (CTC) framework to generate adversarial fork scenarios, including branches that violate expected consensus behavior.
- Develop test cases that simulate Genesis-specific edge cases and adversarial conditions to improve test coverage.
- Enable early detection of consensus flaws by validating node behavior against adversarial fork scenarios.
- Integrate these tests into existing testing workflows to ensure continuous validation of the consensus implementation.

Expected Value

This proposal improves the safety of the Cardano consensus layer by enabling systematic testing of adversarial fork scenarios, including those that expose edge cases in Genesis and related mechanisms. By validating node behavior under adversarial conditions prior to deployment, it enables early detection of consensus flaws that could otherwise lead to incidents on mainnet.

This directly supports network reliability by reducing the likelihood of consensus failures and unsafe upgrades (**Monthly uptime**). By making these tests available across implementations, the proposal also supports client diversity and operational resilience, ensuring consistent behavior across node clients.

Metric of Success

- No identified flaky tests remain in the Genesis test suite, with all tests demonstrating stable results across repeated CI executions.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T6.0 Project Kickoff	Public project setup including repository, documentation, and tracking. Initial description of adversarial fork testing goals and scope. Public announcement/demo introducing the project.	Repository and project materials are publicly accessible. Scope and objectives of adversarial fork testing are clearly documented. A public demo or announcement is available.	2
T6.1 Adversarial Fork Generator Design	Design of enhancements to: - block tree generator (supporting adversarial branching).	The design is accepted by the consensus team.	12

	- point schedule generator (timing of block propagation). Documentation describing how adversarial scenarios are constructed.		
T6.2 Implementation of Adversarial Fork Generation	Implementation of: - adversarial branch generation in block tree generator. - updated point schedule generator supporting complex fork scenarios. Initial integration into testing framework.	Changes are reviewed and accepted for merge as pull requests to the ouroboros-consensus repository. All review feedback is addressed. Generator produces adversarial fork scenarios as defined in the design. Initial tests execute (even if not all pass yet).	8
T6.3 Test Stabilization and Coverage Validation	Stabilization of all tests using adversarial fork scenarios. Analysis of test coverage and identification of previously untested cases. Improvements to ensure deterministic and reliable test execution.	All tests pass consistently across repeated CI runs. Test coverage analysis is documented and published. All changes are reviewed and accepted for merge in the ouroboros consensus repository.	14

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development (Labor)	FTE	12	₳ 117,334	₳ 1,408,008

Supporting resources

Full proposal: https://drive.google.com/file/d/1Cw1TUL4U6232zuzByZJbLvyZWCupl7as/view

Canonical ledger state maintenance and Mithril integration

Area: Operational resilience / Reliability
Core KPI: Monthly uptime, Alt. Full Node Clients

High level description

During the 2025-2026 Treasury Withdrawal period, Tweag focused on the Canonical Ledger State project, which culminated in the development and implementation of CIP-0165. This project aims to introduce a canonical, extensible, and versioned format for the ledger state, enabling reuse across various projects. Additionally, a black-box testing project was undertaken, facilitating the extraction of ledger tests from one node for application to other nodes.

This project is a continuation of the preceding work, with the goal of integrating the canonical ledger state with Mithril. Achieving this necessitates the drafting of an additional CIP to precisely define how nodes should furnish snapshots and to outline future support for snapshot generation. Furthermore, the project involves actual integration with Mithril by utilizing the SCLS format within the snapshot-converter, encompassing verification, signing, and subsequent conversion to the format compatible with the Amaru node.

Core objectives

- Integrate canonical ledger state tooling into the Cardano and Mithril codebases to enable standardized state export and consumption.
- Extend and maintain support for Dijkstra-era ledger state within the canonical ledger state tooling.
- Define and submit a Cardano Improvement Proposal (CIP) specifying how nodes generate canonical state files so they can be accepted and processed by the Mithril network.

Expected value

This proposal enables a standardized representation of the canonical ledger state, allowing nodes and external systems to generate and consume state snapshots in a consistent and interoperable way. By defining a CIP and integrating it with Mithril, it supports the development of alternative node clients and light-client infrastructure.

This reduces integration complexity across implementations and supports client diversity and operational resilience by ensuring consistent handling of ledger state across the ecosystem.

By providing a well-defined interface for accessing ledger state, the proposal also simplifies the development of external tools, such as wallets, analytics, and data extraction services. This may contribute to improved user experience and broader ecosystem adoption (**MAU**), as well as more reliable infrastructure built on top of Cardano.

Metrics of success

- Mithril generates and verifies canonical ledger state snapshots using the SCLS format in a pre-production or production environment.[2]
- At least one alternative node implementation (e.g., Amaru) successfully bootstraps from an SCLS snapshot and reaches a valid ledger state consistent with the network.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T7.0 Project kick off	Public repository and documentation hub for the project. Initial description of canonical ledger state integration with Mithril. Public demo or announcement outlining goals and scope.	Repository and documentation are publicly accessible. Project scope and integration goals are clearly described. A public demo is available on the project website.	2
T7.1 Requirements and Integration Design	Document defining requirements for: - canonical ledger state usage. - Mithril snapshot integration. Agreed design for how nodes produce and consume canonical state.	Requirements and design documents are reviewed and agreed by consensus and Mithril teams. All major integration points are clearly defined. The document is published and accessible.	4
T7.2 Implementation of Canonical State Integration	Implementation of snapshot conversion using canonical ledger state (SCLS format).	Implementation is reviewed and accepted by external reviewers. Snapshots are successfully generated and processed using SCLS format.	24

[2] As Tweag does not own the Mithril codebase we will count as a success the public demo of the integration process.

T7.3 Standardization of Canonical State Generation (CIP)	Cardano Improvement Proposal (CIP) defining: - how nodes generate canonical state files - requirements for interoperability with Mithril Supporting documentation for ecosystem adoption	CIP is submitted and reviewed by CIP editors. Proposal reaches proposed status. A public demo is available on the project website.	4

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development	FTE	15	₳ 117,334	₳ 1,760,010
Development (maintenance)	FTE	6	₳ 117,334	₳ 704,004

Supporting resources

Full proposal:
https://drive.google.com/file/d/1p0REpPeR0RWvoawXOERfE0UNcs9-wWEz/view

Plutus-Script Re-Executor Extension: Ecosystem Integration

Area: Developer experience / Open-source incentives / Education & Migration
Core KPI: MAU, Monthly uptime, Monthly transactions

High level description

Debugging live Plutus script execution on the Cardano blockchain is difficult for several reasons: the deployed on-chain scripts are typically highly optimised to reduce on-chain execution costs and they are executed in the ledger state, which is hard to construct by hand and ephemeral in the cardano-node. There is a demand for tools which assist DApp developers to be able to debug the values computed within a script against the ledger state on-chain and generate test values for scripts.

This proposal would deliver a set of extensions to the Plutus Script Re-Executor (PSR) — a tool for monitoring on-chain scripts that does not maintain its own copy of the ledger state but uses the “leashed” cardano-node, ensuring all transactions are observed while the ledger state is still accessible from the node. Such extensions would improve the debugging infrastructure of the Cardano tools lowering the resources requirement for developers (such as RAM), create new debugging scenarios using the saved scripts environments (same settings and arguments aka the correct ScriptContext as it was executed on-chain), and increase PSR’s eco-system

integration, by introduction of the simple UI interface, cli tools and cookbook explaining how to use the tool for the most common workflows.

Core objectives

- Integrate the Plutus Script Re-Executor extension into the Cardano ecosystem and relevant developer workflows.
- Develop developer-facing tooling for debugging and analysis, including a user interface and tools for managing and substituting scripts.
- Provide comprehensive documentation to support developer adoption and effective use of the re-execution tooling.

Expected Value

This proposal improves the developer experience by enabling reproducible execution and debugging of Plutus scripts. By providing tools to analyze and re-execute scripts, it reduces the time required to identify and fix issues, and lowers the barrier to developing and maintaining smart contracts on Cardano.

Improved debugging and reproducibility lead to more reliable applications and faster development cycles. This may support increased developer activity and application quality, contributing indirectly to user engagement (**MAU**) and **transaction** activity.

By improving the reliability of smart contracts and reducing the risk of errors in production, the proposal may also strengthen confidence in the ecosystem, potentially contributing to **TVL** growth.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T8.1 Tool Specification and Design	CIP is submitted and reviewed by CIP editors. Proposal reaches accepted status.	Specification is published in the repository. All planned features and workflows are clearly described.	4

	Specification is complete and usable by external teams.	Proposed changes are reflected in planned pull requests.	
T8.2 Web Interface for Script Monitoring and Analysis	Web application enabling: - monitoring of live DApp script execution. - inspection of script inputs, outputs, and execution context. Integration with PSR backend for real-time data access.	The web interface is functional and connected to live or recorded execution data. Users can inspect script execution details through the UI. Public demo showcases end-to-end usage of the interface.	12
T8.3 Runtime Script Substitution Management	Additional endpoints together with frontend UI functionality added to the PSR HTTP API which would allow for CRUD modifications of target and shadow scripts, developed as PRs against the PSR.	CRUD operations for scripts are available via API and UI. Changes take effect without restarting the system. Public demo showcases end-to-end usage of the interface.	4
T8.4 Script Re-Execution with Modified Contexts	Additional endpoints added to the PSR HTTP API which would allow running scripts against modified ScriptContexts.	Scripts can be executed against stored contexts with modified parameters. Results are returned and inspectable via API or UI. Implementation is delivered as reviewed pull requests.	4
T8.5 Export and Reuse of Execution Data	Additional endpoints added to the PSR HTTP API which would allow exporting saved data; Additional executables can be provided for similar purposes.	Execution data can be exported in a usable format. Exported data can be reused outside the tool.	4
T8.6 Test Suite and Validation Infrastructure	A robust test suite. Potentially also: requests to the Consensus Team and/or Node Team to tweak their libraries' stable interface.	Tests are implemented and merged into relevant repositories. All core functionality is covered by automated tests.	16

		Tests pass consistently in CI.	
T8.7 Developer Documentation and Ecosystem Integration	Public Cardano user resources will include use case scenarios, a cookbook, and integration instructions for the tool with other debug utilities (e.g., cooked-validator), Docker containers are provided, etc.	Documentation is complete and publicly accessible. The tool can be installed and used following documentation only. Documentation is reviewed and accepted by relevant teams.	6

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Cost of the project	PSR project	1	₳ 2,500,00	₳ 2,500,000
Development	FTE	36	₳ 117,334	₳ 4,224,024

Supporting resources

Full proposal: https://drive.google.com/file/d/1mTn9OTq4r890BoDRsi5GPNZ2piOA0NIa/view

Mutation Testing for Cardano

Area: Reliability
KPI: Monthly uptime

High level overview

Ensuring the long-term reliability and robustness of the Cardano node implementation is paramount for the stability and security of the entire blockchain. While the existing development process utilizes rigorous testing, including Property-Based Testing (PBT), there remains a critical need for an automated, objective metric to gauge the **completeness and effectiveness** of the current test suite. A test suite that passes, yet fails to detect a trivial code modification (a *mutation*), indicates a significant "blind spot". That is, a potential regression that could silently pass through the CI/CD pipeline and lead to critical, unintended behavioral changes in a deployed release.

This proposal focuses exclusively on the **Test Suite Adequacy** problem, and advocates for the implementation of the first iteration of a systematic **Mutation Testing** framework for the Cardano codebase. The goal is to automatically introduce small, representative faults (mutations) into the Cardano's codebase and then verify whether the existing test suite successfully detects them (i.e., whether any of the existing tests fail). The resultant metrics, including a **Mutation Score**, will

serve as quantifiable measures of test suite quality, guiding developers toward inadequately tested areas of the code and proactively preventing future regressions.

Core objectives

- Develop a mutation testing framework for Haskell, including a code mutator that generates controlled variations of source code.
- Implement a mutation execution harness to run mutated code against existing test suites.
- Define and generate test adequacy metrics (e.g., mutation score) to evaluate test coverage and effectiveness.
- Apply the framework to the Peras codebase to assess and improve test quality.
- Provide developer documentation to support adoption and effective use of the mutation testing framework.

Expected value

This proposal introduces mutation testing as a method to evaluate the effectiveness of existing test suites by identifying missing scenarios and uncovering subtle bugs that are not detected by traditional coverage metrics. By systematically testing the robustness of existing tests, it improves confidence in code correctness across the Cardano codebase.

Applying this approach to the Peras codebase strengthens test quality for a critical consensus component, reducing the risk of defects reaching production. This directly supports network reliability and safer protocol evolution (**Monthly uptime**).

As a reusable testing technique, mutation testing can be applied across multiple components of the ecosystem, improving **overall code quality** and **reducing systemic risk**. Indirectly, this supports safer deployment of scalability improvements (e.g. Peras), contributing to **throughput capacity per day** and ecosystem confidence (**TVL**).

Metrics of Success

- Mutation testing framework successfully generates and executes mutations against Haskell codebases, with results reproducible across runs.
- Mutation testing is applied to at least one core Cardano package, with results generated and reported as part of the development or CI workflow.
- Mutation score for the Peras codebase increases from baseline and reaches a defined threshold (e.g., ≥ 70%), demonstrating improved test effectiveness.

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T9.0 Project kickoff	Public repository with initial project structure and documentation. Defined scope and plan for mutation testing framework. Development environment enabling reproducible experimentation.	The repository is publicly accessible with setup instructions. Project plan and next steps are documented. The environment can be used by others to reproduce initial setup.	2
T9.1 Mutation Engine Proof of Concept	Implement a GHC compiler plugin capable of applying basic mutation operators (e.g., arithmetic swap, relational swap) to a single Haskell module, generating mutated implementations.	Mutation engine produces valid mutated code variants. Mutations can be applied to at least one Cardano module. Public demo shows reproducible mutation generation and execution.	6
T9.2 Automated Mutation Execution and Scoring	Implement an automated toolchain to compile, run tests, and capture results for a mutated codebase; successful calculation of a preliminary Mutation Score.	Mutated code can be automatically tested end-to-end. Mutation score is computed and reported for selected components. Public demo shows reproducible execution and scoring.	6
T9.3 Expanded Mutation Coverage and Reporting	Implement 10+ robust mutation operators, evaluating the feasibility of other advanced mutation/orchestration techniques. Finalize the reporting database and generate initial Mutation Score reports on selected components.	The mutation engine supports multiple mutation types. Reports clearly identify surviving mutations (undetected issues). Public demo presents mutation score results and findings.	4
T9.4 CI Integration and Continuous Quality Monitoring	Integrate the mutation testing framework into the selected Cardano CI/CD pipeline for continuous quality assessment. Integration will initially target nightly runs on selected components	Mutation testing runs automatically within a nightly CI/CD pipeline.	4

	rather than full mutation coverage across the codebase. Effectiveness will be evaluated by comparing mutation survivability before and after targeted test-suite improvements derived from mutation reports.	Results are consistently generated and accessible. Improvements in mutation score are demonstrated over time.	
T9.5 Documentation and Final Evaluation Report	Deliver documentation of the Mutation Testing framework, including user guides for developers. Deliver comprehensive Mutation Score report detailing the results achieved and recommended next steps for optimization.	Documentation is complete and publicly accessible. Mutation testing results are reproducible. Final report clearly explains findings and next steps.	2

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development	FTE	12	₳ 117,334	₳ 1,408,008

Supporting resources

Full proposal: https://drive.google.com/file/d/1X6AX7eTRsTELKhJzaEKts601PyjS5p8X/view

Hoarding node: Live Network Deployment

Area: Reliability
Core KPI: Monthly uptime

High level description

The hoarding node has been developed and validated in local testing environments. To collect real observability data from the Cardano network, it needs to run continuously against a live network — connected to real peers, exposed to real traffic, and accumulating data over time.

This work package covers the deployment infrastructure and live operation of the hoarding node against the Cardano pre-production and mainnet networks. It includes the full deployment pipeline — AWS infrastructure provisioned as code, automated deployments from CI, encrypted secrets management, and automated database migrations — as well as six months of live operation on preprod/mainnet.

The deployment is built entirely from infrastructure-as-code and serves a dual purpose: it is the team's own live environment for collecting real data, and it is the reference implementation that external operators can follow to run their own instance. External operators need only the NixOS module or OCI image and a way to provide secrets as environment variables — no separate infrastructure repository or tooling required.

Deliverables:

- Terraform, NixOS configuration, and CI/CD pipeline for the team's deployment.
- nixosModules.hoard flake export and OCI image, documented for external operators.
- Six months of live operation against preprod, with observability dashboards.

Core Objectives

- Deploy the hoarding node to continuous operation on Cardano pre-production and mainnet networks, collecting real blocks, headers, and peer data over an extended period.
- Implement fully automated, reproducible deployment infrastructure (infrastructure-as-code and CI/CD pipelines) requiring no manual intervention after initial setup.
- Provide deployment artifacts (e.g., NixOS modules, OCI images) enabling external operators to run hoarding node instances with minimal setup.
- Deliver observability tooling (dashboards and monitoring) to provide continuous visibility into network data collection, peer connectivity, and system health.
- Provide user documentation including quickstart guide and API usage cookbook with queries examples.

Expected Value

This proposal enables continuous observability of the Cardano network by deploying the hoarding node on pre-production and mainnet, allowing detection and analysis of anomalous and adversarial behaviour that cannot be captured through local testing alone.

By providing sustained visibility into block propagation, peer behaviour, and invalid or orphaned blocks, the proposal supports improved threat detection and operational reliability (**Monthly uptime**).

Making the deployment accessible to external operators increases the number of independent observation points, improving coverage and data quality, and strengthening the network’s ability to detect and respond to issues in a timely and decentralized manner.

This proposal establishes the data foundation required for subsequent validation, transaction collection, and distributed observability work.

The hoarding node acts as a safety layer during the scaling phase enabled by Leios and Peras.

Metrics for Success

- Hoarding node operates continuously on pre-production for ≥ 99% of the observation period, with observability dashboards available.
- Orphaned and invalid block data is continuously collected and queryable via the HTTP API, with data availability ≥ 99% over the observation period.
- The system successfully detects and classifies anomalous or invalid blocks in controlled test scenarios, with results observable and queryable via the HTTP API.
- If transaction collection is included: mempool transactions are collected, stored, and queryable end-to-end via the HTTP API.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T10.1 Deployment Pipeline	Cloud resources provisioned via infrastructure-as-code. Automated CI/CD pipeline deployment.	A code push triggers a complete deployment to a running environment Infrastructure (compute, database, networking) is provisioned automatically. Secrets are managed securely and never exposed in plaintext. Failed deployments automatically roll back without leaving the system in an inconsistent state.	4
T10.2 Live Deployment	Hoarding node running continuously against the Cardano network, exposing data via API for consumption. Observability stack functioning, with dashboards showing operational metrics.	Node is actively connected to the network and collecting real data. Orphaned and invalid block data is accessible via the HTTP API. Observability dashboards are live and provide real-time visibility. Monitoring and alerting are configured and functioning.	2
T10.3 External Operator Artifacts	NixOS module and OCI/Docker image made available as deployment paths for operators.	An external operator can deploy and run the hoarding node using only published artifacts and documentation.	2

		Deployment does not require access to internal infrastructure or custom tooling. The setup process is reproducible and validated end-to-end.	

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Implementation of hoarding node during 2025-2026	Work cost	1	₳ 2,500,000	₳ 2,500,000
Development (Labor)	FTE	4	₳ 117,334	₳ 469,336
Cloud cost (preprod)	EC2: t3.xlarge (4 vCPU, 16GB) EBS: 100GB gp3 RDS: db.t3.medium, single-AZ RDS storage: 50GB gp3 Data transfer + misc	12	₳ 500	₳ 6,000
Cloud cost (mainnet)	EC2: t3.xlarge (4 vCPU, 16GB) EBS: 100GB gp3 RDS: db.t3.medium, single-AZ RDS storage: 50GB gp3 Data transfer + misc	12	₳ 500	₳ 6,000

Supporting resources

Full proposal: https://drive.google.com/file/d/1JfSZqTYckt462Cz5CPs2qcHlq-b9WgRk/view

Hoarding node: Distributed Mode

Area: Reliability
Core KPI: Monthly uptime

High level description

The hoarding node is an observer on the Cardano network. It connects to multiple Cardano peers simultaneously, collects blocks and block headers as they propagate, and records them in a database. This data enables detection of adversarial behaviour — such as blocks produced by illegitimate slot leaders, conflicting chain forks, or invalid transactions — as well as analysis of how data propagates across the network over time.

In its current form, the hoarding node runs as a single process in a single location. This limits its ability to measure how quickly data reaches different parts of the network — an observer in one region cannot capture propagation timing from other vantage points across the global network.
This work package implements distributed mode: a deployment architecture in which multiple lightweight collectors, running in different geographical regions, forward their observations to a central coordinator for processing and storage. The collector is responsible only for connecting to Cardano peers and forwarding what it sees. The coordinator receives data from all collectors, runs the processing pipeline (anomaly detection, classification, persistence), and exposes it via the HTTP API.

Collector-coordinator communication is built on the same typed-protocols, network-mux, and ouroboros-network-framework stack that Cardano itself uses for peer-to-peer communication. This is not a custom messaging layer — it reuses proven infrastructure from the Cardano node codebase.

The existing single-node deployment mode is fully preserved. Operators who do not need multi-region coverage continue to run a single hoard binary with no configuration changes. Distributed mode is an opt-in deployment topology, not a replacement.

Deliverables:

- Three executables: hoard-collector, hoard-coordinator, and hoard (single-node, combining both roles).
- A typed protocol layer covering the operations required in distributed mode (for example, peer assignment).
- NixOS modules and OCI images for both roles, deployable independently.
- Integration tests covering both happy-path and coordinator-disconnect scenarios.

Core Objectives

- Enable multi-region deployment of the hoarding node, allowing simultaneous observation of the network from multiple geographical locations.
- Implement measurement of block and header propagation timing across regions, providing data on network latency and propagation behavior.
- Design and implement a distributed architecture with a clear separation between data collection (collectors) and processing (coordinator).
- Maintain compatibility with existing single-node deployments, preserving non-distributed operation as a supported mode.

Expected Value

This proposal enables multi-region observation of the Cardano network, allowing measurement of block and header propagation across geographically distributed nodes. This provides visibility

into propagation delays and network behavior that cannot be obtained from a single-node deployment.

By producing structured propagation timing data, the system improves the ability to detect network anomalies, delays, and inconsistencies. This supports improved threat detection and operational reliability (**Monthly uptime**) by enabling earlier identification of network issues.

The distributed architecture also enables broader and more consistent network coverage, strengthening observability and supporting research and infrastructure efforts aimed at improving network performance and resilience.

This proposal provides a structured, multi-regional dataset of network propagation behavior, which is currently not available in a consistent and publicly accessible form."

Metrics for Success

- At least 2 collectors are deployed in distinct geographical regions and connected to a single coordinator.
- ≥ 99% of observed blocks and headers are successfully ingested and stored in the hoarding database.
- Propagation timing data is recorded for ≥ 95% of observed blocks across all active collectors.
- Propagation delay between regions is measurable and available for analysis over the observation period.
- Single-node mode maintains ≥ 99% uptime with no regression in behavior compared to baseline deployment.
- Collector–coordinator communication achieves ≥ 99% successful reconnection rate after disconnection events during testing.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks

T11.1 Collector/Coordinator Split	The codebase is restructured into clean collector and coordinator modules with distinct effect stacks.	All three binaries (collector, coordinator, single-process) are built separately and the hoard single-node binary passes all existing tests with behaviour identical to the pre-split version.	2
T11.2 Protocol Layer and Single-node Mode	The collector-coordinator protocol layer is implemented using the ouroboros-network-framework.	Hoard operates end-to-end using the new protocol layer with no regression in behaviour. Integration tests cover the full protocol exchange, including coordinator-disconnect scenarios, and latency/packet-loss scenarios.	3
T12.3 Distributed Mode and Deployment	Collector and coordinator deployable as separate networked processes, communicating over the network, with the collector connecting to a configured remote coordinator address. All three executables are packaged with NixOS modules and OCI images.	Two separate collectors connect to a single coordinator over the network; data from both are aggregated and exposed via the API. NixOS modules and OCI images are documented and usable by external operators.	3

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Resources (Labour)	FTE	4	₳ 117,334	₳ 469,336

Supporting resources

Full proposal: https://drive.google.com/file/d/1LtHfh3SgjCG-xzgvqGIU2x6VYvdoQHty/view

Hoarding node: Embedded Consensus Validation of Blocks and Headers

Area: Reliability
Core KPI: Monthly uptime

High level description

The hoarding node connects to multiple Cardano peers simultaneously and records the blocks and headers they advertise. It currently performs a set of structural checks on each block — detecting integrity failures, mismatches between headers and bodies, and blocks outside the requested range. These checks require only the block itself and catch a class of obvious anomalies.

What they cannot catch is consensus and ledger invalidity: a block that is structurally well-formed but was produced by a peer who was not the legitimate slot leader, or one containing

transactions that violate UTxO rules. Detecting this class of invalidity requires the full ledger state — knowledge of who was elected to produce each slot, what the current stake distribution is, and what transactions are valid given the current UTxO set.

This work package embeds ouroboros-consensus — the same consensus library used by the Cardano node — directly into the hoard process. Rather than querying an external Cardano node over a socket, hoard runs the full validation pipeline in-process. This expands block classification from a binary (canonical / orphaned) to a richer taxonomy that includes three stages of invalidity: header envelope errors, consensus errors (invalid slot leader claim, bad VRF proof), and ledger errors (invalid transactions).

A second capability follows from this: header validation. Block headers arrive via ChainSync before block bodies are fetched. With access to the ledger state, hoard can validate each header as it arrives — detecting an invalid slot leader claim at the earliest possible moment and attributing it to the specific peer that advertised it, without ever downloading the block body.

As a by-product, the external Cardano node is no longer required as an operational dependency. The two things hoard currently queries from it — the immutable tip and per-block canonical status — are both provided directly by the embedded consensus store.

Deliverables:

- Richer block classification, including three stages of invalidity, recorded in the hoarding database and exposed via the HTTP API.
- Per-peer header validation at the ChainSync stage, before block bodies are fetched. Invalid headers recorded and attributed to the peer that advertised them.
- Removal of the external Cardano node as a required operational dependency.

Core Objectives

- Implement embedded validation of blocks and headers within the hoarding node to detect consensus and ledger invalidity across validation stages (header envelope, consensus, and ledger).
- Enable classification and recording of invalid blocks and headers as indicators of anomalous or adversarial behavior.
- Attribute invalid headers detected during ChainSync to the originating peer, enabling early and precise identification of potentially malicious actors.
- Remove the dependency on an external Cardano node by integrating required validation logic directly into the hoarding node.

Expected Value

This proposal enables the hoarding node to perform embedded consensus and ledger validation, allowing detection of blocks and headers that violate Ouroboros rules, including invalid slot leadership and UTxO inconsistencies. This introduces a new capability to identify and classify adversarial behaviour that is not detectable through structural validation alone.

By attributing invalid headers to specific peers and detecting invalidity at early stages, the system provides a verifiable and queryable record of anomalous network activity. This improves the ability to detect and analyze threats in real time.

These capabilities directly support improved network reliability by increasing visibility into consensus-level issues and potential attacks (**Monthly uptime**). Additionally, removing the dependency on an external Cardano node simplifies deployment and reduces operational overhead.

Metrics for Success

- ≥ 95% of injected or observed invalid blocks (consensus or ledger violations) are correctly detected and classified across validation stages, as measured in pre-production or controlled test scenarios.
- Detected invalid blocks and headers are available via the HTTP API with ≥ 99% query success rate over the observation period.
- ≥ 95% of invalid headers detected at the ChainSync stage are successfully attributed to the originating peer.
- Detection and classification of adversarial scenarios (e.g., invalid slot leader claims, UTxO-invalid transactions) is demonstrated in controlled or pre-production environments, with results reproducible and queryable.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks

T12.1 Embedded ChainDB Integration	The embedded ChainDB runs inside the hoard process and is fed blocks as they arrive. Existing functionality is unchanged.	The ChainDB syncs in parallel with the existing Cardano node. New events appear in logs confirming blocks are being validated and chain updates observed.	2
T12.2 Block Validation	The ChainDB is used to classify blocks. Invalid blocks are stored with their validation stage and error detail.	Invalid blocks are classified by stage and stored in the database. Existing canonical/orphaned classification is unaffected.	2
T12.3 Header Validation	Validate each incoming against the embedded ChainDB's ledger state. Invalid headers are recorded and attributed to the peer that advertised them.	Invalid headers (envelope and consensus stage) are detected, recorded, and attributable to a specific peer. Valid headers pass through without disruption. Headers beyond the forecast horizon are logged at debug level and skipped.	3
T12.4 API Exposure	Richer block classification (including all invalidity stages) and invalid headers are exposed via the HTTP API.	All three stages of block invalidity are queryable via the API. Invalid headers are queryable and attributable to specific peers.	1
T12.5 Remove External Node Dependency	Pre-existing analysis such as the orphaned block detection is switched to use events from the ChainDB. The external Cardano node is no longer required.	The hoarding node runs end-to-end without a Cardano node socket. Canonical and orphaned classification continues to work correctly via ChainDB events. All tests pass. Documentation no longer references the external node as a required component. Demo publicly accessible to the community via a shared link and recorded on the project website	1

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Resources (Labour)	FTE	4	₳ 117,334	₳ 469,336

Supporting resources

Full proposal: https://drive.google.com/file/d/1DtU1Sr3pQoKIs23srNYf3v5gideTQWVG/view

Hoarding node: Transactions collection

Area: Reliability
Core KPI: Monthly uptime

High level description

The hoarding node connects to multiple Cardano peers simultaneously and records the blocks and headers they advertise. It does not currently observe mempool transactions: the transactions peers hold in their mempools, not yet included in any block, are invisible to the node.

Collecting mempool transactions requires running the TxSubmission2 mini-protocol in server (responder) role — peers push transaction IDs to the server, which pulls the full transactions. Transaction collection was deferred from the first cycle due to complexity and the absence of a live deployment; at the time, the prevailing understanding was that the server role requires accepting inbound connections from publicly reachable peers.

Since then, a further path has been identified. The Ouroboros network specification (§5) describes duplex connections: a single TCP connection can carry mini-protocols in both directions simultaneously. When the hoarding node advertises InitiatorAndResponderDiffusionMode, remote peers' outbound governors may promote the hoarding node's outbound connections to duplex, enabling them to run TxSubmission toward us on the same connection we opened. No listening socket or publicly reachable deployment is required. A prototype implementing this approach has been built. It has been run against preprod but has not yet witnessed any promoted connections, so it remains unvalidated end-to-end.

If promotion is confirmed not to work reliably in practice, two fallback paths exist: inbound connections via a publicly reachable deployed node (depends on the infrastructure work package), or LocalTxMonitor via a node-to-client connection to a trusted full node.

A note on coverage: with the duplex promotion path, transactions can only arrive from peers whose outbound governor chooses to run TxSubmission toward us. We cannot solicit transactions from peers we connect to — the protocol flows the other way. Coverage is therefore not fully under our control.

Deliverables:

- Mempool transactions received from peers recorded in the hoarding database with attribution to the peer that submitted them.
- Transactions queryable via the HTTP API, filterable by peer and time window.
- A confirmed working path to receive transactions — connection promotion, inbound connections, or LocalTxMonitor — with the chosen approach documented.

Core Objectives

- Implement collection of mempool transactions from connected peers, enabling observation of transactions prior to block inclusion with peer attribution.
- Validate whether outbound connections can be promoted to duplex in practice, confirming feasibility for transaction collection.
- Persist collected transactions in the hoarding database and expose them via the HTTP API with associated peer information.

Expected Value

This proposal enables collection of mempool transactions, providing visibility into network activity before transactions are included in blocks. This fills a key observability gap between transaction submission and block inclusion.

By capturing transaction propagation, duplicate submissions, and peer-attributed activity, the system enables analysis of transaction flow and early detection of anomalous behavior. This improves visibility into network dynamics and supports faster identification of issues affecting transaction processing.

These capabilities contribute to improved operational reliability by enabling earlier detection of network disruptions and abnormal transaction patterns (**Monthly uptime**). Additionally, analyzing transaction propagation and inclusion behavior supports understanding of network performance and effective utilization of throughput capacity.
If outbound connection promotion is confirmed, transaction collection can be performed without additional infrastructure, **reducing deployment complexity** and improving accessibility for operators.

Metrics for Success

- ≥ 95% of observed mempool transactions from connected peers are successfully received and stored in the hoarding database.
- Stored mempool transactions are queryable via the HTTP API with ≥ 99% success rate over the observation period.
- ≥ 90% of blocks contain at least one transaction previously observed in the collected mempool dataset.
- ≥ 95% of collected mempool transactions are attributed to the originating peer.
- Outbound connection promotion is validated by measuring the percentage of connections successfully upgraded to duplex (target ≥ 60%), or a documented fallback approach is implemented and verified.

Strategic Alignment

Pillar: Pillar 1 — Infrastructure & Research Excellence (Focus Area I.2: Threat Detection & Recovery)
Rationale: Mempool visibility completes the network observability picture. Blocks record what was accepted; transactions record what was attempted. Anomalous transaction patterns — spam, invalid submissions, coordinated flooding — are detectable at the mempool stage before they affect block production.
KPI support: Expands the range of observable network events, supporting both threat detection and research into transaction propagation dynamics.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T13.1 Connection Promotion Investigation	A clear answer to whether outbound connections are promoted to duplex in practice. If they are: this milestone delivers a working transaction collection pipeline. If they are not: a diagnosis of why and a documented decision on the fallback path.	Either: at least one promoted connection observed and transactions received, confirming the duplex path works. Or: a written diagnosis explaining why promotion does not occur in practice, with a decision to proceed via inbound connections instead.	2
T13.2 TxSubmission Pipeline	The transaction collection pipeline is wired into persistence with associated metadata. If Milestone 1 identifies that peers are rotating us out, appropriate measures are implemented to remain a viable hot peer candidate.	Transactions received from peers appear in the database. Per-peer attribution is correct. Duplicate transactions from different peers are deduplicated by transaction ID.	3
T13.3 API Exposure	Collected transactions are queryable via the HTTP API. Endpoints support filtering by peer, time window, and transaction ID.	Transactions are queryable via the API. Results are attributable to specific peers. Demo publicly accessible to the community via a shared link and recorded on the project website	1

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Resources (Labour)	FTE	3	₳ 117,334	₳352,002

Supporting resources

Full proposal: https://drive.google.com/file/d/1NbpyLDYaIXoAI2n6Re9sL23YadIMNo8C/view

Block Cost Investigation Extensions

Area: Revenue / adoption
Core KPI: Annual Protocol Revenue

High-level description

The parameter values of Ouroboros Praos for its specific instantiation on Cardano mainnet were tuned many years ago. One such parameter is Δ, the worst-case delay for a fresh honest block to be reliably adopted by the producer of the next honest block. The transactions and blocks have become much more sophisticated since those tuning decisions, most notably with Plutus scripts in the Alonzo era and reference inputs in the Babbage era. It is overdue to investigate whether the adversary might be able to mint a worst-case block that is so resource intensive for honest nodes today to validate that the actual worst-case message delay is much greater than the values used so far for Δ.

Recent relevant examples include the June 2024 attack and a performance bug detected shortly after the Chang hard fork to the Conway era (the potential DoS vector mentioned in the 10.1.4 release notes). Could a more sophisticated adversary have done much more harm?

Core objectives

- Analyze historical blockchain data to identify correlations between protocol parameters and transaction execution costs.
- Develop a data-driven transaction cost model based on observed execution patterns.
- Implement a regression analysis test suite to validate the accuracy and stability of the cost model.
- Integrate the model into runtime analysis of mainnet data to evaluate transaction costs in real-world conditions.

Expected Value

This proposal improves the accuracy of transaction and block cost estimation by analyzing real execution data and identifying discrepancies between expected and observed costs. This

reduces the risk of misconfigured parameters and inaccurate cost assumptions.

By developing a data-driven cost model and regression testing framework, the project enables early detection of regressions in transaction cost behavior before they reach mainnet. This supports safer protocol evolution and more reliable network operation (**Monthly uptime**).

Improved cost estimation also enables better utilization of block capacity, supporting scalability and more efficient transaction processing (**Throughput capacity per day**).

Metrics of Success

- Prediction error of the cost model is measured and reported against observed mainnet data, including distribution metrics (mean, median, p95).
- Prediction error is reduced compared to the existing cost estimation approach, based on analysis of historical data.
- Regression test suite detects deviations in transaction cost behavior relative to established baselines, with results reproducible across runs.
- The cost model is validated against a representative dataset of mainnet transactions spanning multiple epochs, with results documented.
- The model is applied to live or recent mainnet data, producing continuous cost estimates over the observation period without interruption.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
T14.1 Empirical model and fitting	Report explaining the empirical cost model and its derivation from mainnet blocks.	The report is published and publicly accessible. The model is derived from real mainnet data (dataset referenced and reproducible). Methodology for data collection and fitting is clearly described.	8

		The model produces cost estimates for a representative sample of transactions. Results include error analysis (e.g., mean / median / p95 deviation vs observed costs).	
T14.2 Analytical model and report	Report explaining the theoretical cost model and its derivation from the specifications. Report explaining a general method for revising and/or extending those models as the ledger rules and consensus protocol evolve.	The report is published and publicly accessible. Analytical model is formally defined and traceable to protocol specifications. Assumptions and limitations are explicitly documented. The model can be used to compute cost estimates for defined transaction scenarios. Method for extending or updating the model is clearly described and reproducible.	4
T14.3 Detectors based test-suite	Transaction generator that can evaluate and benchmark engines to find regressions.	Transaction generator produces reproducible test cases. Benchmarking tool executes generated transactions and records results. Test suite detects deviations from expected cost behavior. At least one regression scenario (possibly artificial) is demonstrated and detected Tooling is integrated into a runnable test environment (CLI or CI-compatible) and demonstrated on publicly available demos.	4
T14.4 Detectors and hoarding node integration	Specification and implementation of automatic anomaly detectors for mainnet. Specification and implementation of regression suites for use in Continuous Integration, both via empirical mainnet blocks and also synthetic	Anomaly detection rules are implemented and executable on real or recorded mainnet data	4

	block content.	Regression test suite runs in CI and produces consistent results Detectors operate on both: - empirical mainnet data - synthetic/generated data	

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Development	FTE	4	₳ 117,334	₳ 469,336

Genesys Sync Accelerator project maintenance

Areas: Reliability
Core KPI: Monthly uptime

High-level description

During the Tweag's multiple infrastructure projects proposals 2025, Tweag implemented multiple core projects, and those projects require additional maintenance, such as:

1. **Maintaining the backlog and roadmap**
    - Develop and publicise a medium-term roadmap for the development of the packages emulator, based upon the results of planning, community discussion and tracked issues.
    - Based upon this roadmap, we will maintain a short-term backlog of work items.
2. **Reviewing and Merging PRs**
    - Monitor the active PR lists and shepherd them to completion.
    - Assist external contributors in making contributions.
3. **Stakeholder and Community Engagement**
    - Identify and maintain the list of key stakeholders for the project
    - Create channels for contributors to engage with developing the backlog and priorities for the project.
    - Identify work items particularly suitable for work on by newcomers and external contributors.
    - Regularly inform stakeholders about the state of the project.
4. **Establishing clear project governance**

- Establish project governance structures according to the Intersect governance framework (https://intersect.gitbook.io/open-source-committee/policies/governance).
- Establish project documentation as per the Intersect documentation framework (https://intersect.gitbook.io/open-source-committee/policies/documentation).

5. **Perform regular maintenance**
   - Work on the backlog items as identified in (1).
   - Keep the project up-to-date with dependencies - in particular, with the current iteration and era of cardano-node.
   - Maintain a CHANGELOG of all major changes to the codebase.

Core objectives

- Provide ongoing maintenance and support for the Cardano node emulator package, including bug fixes, compatibility updates, and integration with new protocol features.
- Provide ongoing maintenance and support for the Genesis sync accelerator project, ensuring compatibility with evolving Cardano node versions and network changes.
- Ensure timely resolution of issues and maintain operational readiness of both components within the Cardano ecosystem.

Expected Value

This proposal ensures continued reliability and usability of critical infrastructure components, including the Cardano node emulator and Genesis sync accelerator. By maintaining compatibility with evolving protocol changes and resolving issues in a timely manner, it prevents degradation of developer workflows and infrastructure performance.

This directly supports network reliability and developer productivity by ensuring that dependent tools and systems remain operational (**Monthly uptime**). It also reduces integration risk for ongoing and future projects relying on these components.

Metrics of Success

- ≥ 95% of reported issues are acknowledged within a defined response time (e.g., 3–5 working days).
- ≥ 90% of critical and high-priority issues are resolved within agreed timeframes.
- Compatibility with the latest Cardano node releases is maintained, with no blocking issues for supported components.
- Maintenance updates (bug fixes, compatibility patches) are delivered and merged on a regular basis, with changes adopted in dependent projects.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
Projects Kickoff	Define roadmap, issue tracker, points for project design, documentation, community resources and reports.	Public demo explaining and introducing the projects, commitments and plans	4

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Maintenance of genesis sync accelerator (Labor)	FTE	2	₳ 117,334	₳ 234,668

Cardano-node-emulator maintenance

Areas: Reliability
Core KPI: Monthly uptime

High-level description

During Tweag's multiple infrastructure projects proposals 2025 Twag updated and maintained cardano-node-emulator (CNE), a tool to emulate the validation of transactions using the actual ledger code, but without a deployed node. This project is critical for audit and testing purposes, as it allows users to submit a limitless amount of transactions in a very short amount of time, to uncover possible issues in smart contracts involved in those transactions.

The work performed in 2025 was threefold:

- An initial update was performed, to align the various projects contained in CNE with the newest version of the dependencies, while also updating some design aspects no longer in line with the current smart contract development standards. In particular, plutus-script-utils, a set of tools to help developers write plutus smart contracts, had been revamped to relegate typed validators to old convenient artifacts for older eras, while introducing multipurpose scripts for plutus version 3 and above.

- Some continuous maintenance was performed, to keep the project aligned with the dependencies, while handling issues, pull requests.
- Some means of communicating with the community has been established, including a dedicated discord channel on the official Intersect discord, as well as an update meeting every other month.

We propose to pursue and increase this effort as part of this proposal.

Core objectives

For the upcoming years, the objectives around CNE are as follows:

- Prepare the release of Dijkstra, study the impact of the changes on CNE, and implement the necessary updates to be ready for the release.
- Keep maintaining the tool, integrating pool requests from the community, engaging with community members on Discord and fixing possible bugs and issues.
- Engage with the community, in a way that makes it visible that CNE can once again be relevant for developers of smart contracts, as it used to be in the past.
- Update heavily outdated parts of CNE. Each individual project will be subject to a relevance study, and updated if applicable.

These objectives will be prioritized and handled based on the total allocation and the discussions with the community.

Expected Value

This proposal is to ensure that, above the initial work performed on CNE last year, the tool remains relevant and up to date with the current standard in developing and testing smart contracts. Above all, alignment and communication with the community will be fostered to ensure any work performed in that direction is supported and used across the Cardano ecosystem. Our goal is to make development of smart contracts safer and CNE is a key part of that endeavor.

Metrics of Success

- Each new issue or pull request is handled swiftly.
- Each of the 4 axes above is given priority, and handled according to that priority.
- Each change is documented and a CHANGELOG is kept up to date.
- CNE is updated to work with the Dijkstra around the time of the official release.

Classification

New initiative or continuation of existing: Continuation
Primary nature: Technical

How this work package will be delivered

Milestones

Milestone	Deliverables	Acceptance criteria	Weeks
Projects meetings	Meetings notes, with a clear depiction of the progress of the past two months.	Public showcasing of the progress during the meeting.	Every 8 weeks

Estimated budget

Cost category	Item Description	Item Quantity	Unit cost	Item total cost
Maintenance of cardano node emulator (Labor)	FTE	20%	₳ 117,334	₳ 469,336

Supporting documents

Full proposal:
https://drive.google.com/file/d/1k1Ktgk3S3qV7CYXrtDmneodySRgfspF5/view